



La Extensión de Jeff es una extensión global e independiente para el agente de codificación pi. Transforma la experiencia predeterminada del agente en un asistente de desarrollo altamente estructurado y autoorganizado mediante la introducción de un “Cerebro” (memoria de trabajo) y “Elementos de conocimiento” (memoria a largo plazo).

Por qué existe esta extensión?

Por defecto, un asistente de codificación de IA comienza cada sesión desde cero, dependiendo totalmente del contexto inmediato que usted proporcione. Para tareas complejas, de múltiples sesiones o arquitectónicas, esto puede llevar a: **Pérdida de contexto:** El agente olvida patrones de diseño o decisiones tomadas en sesiones anteriores. Ejecución no estructurada: El agente podría saltar directamente a modificar el código sin entender completamente la arquitectura o sin obtener su aprobación. Artefactos dispersos: Se generan documentos importantes, como listas de tareas y planes de diseño, pero no se realiza un seguimiento persistente de ellos. Jeff resuelve esto mediante la introducción de:

Modo de planificación: Obliga al LLM a detenerse y formular un plan arquitectónico (con aprobación del usuario) antes de modificar la base de código en tareas complejas. **Conocimiento persistente (KIs):** Permite al agente leer y escribir “Elementos de conocimiento” (metadata.json) entre sesiones, asegurando que aprenda los patrones específicos de su repositorio a lo largo del

tiempo.**Cerebro estructurado:** Dirige automáticamente los documentos de seguimiento (task.md, implementation_plan.md, walkthrough.md) a un directorio dedicado en ~/.jeff/brain/<session-id>/ para que la raíz de su proyecto se mantenga limpia mientras el agente permanece organizado.**Detalles técnicos**La extensión es un módulo de TypeScript cargado dinámicamente por pi al iniciar. Se encuentra en su directorio de extensiones global en ~/.pi/agent/extensions/jeff.ts.

1. Inyección de contexto (beforeagentstart)

La extensión se conecta al evento beforeagentstart. Antes de que el LLM genere un solo token, Jeff modifica el prompt del sistema para inyectar:

Metadatos del espacio de trabajo: Inyecta el sistema operativo actual (os.platform()), la hora local y el nombre de usuario activo para darle al modelo un contexto del mundo real.**Resúmenes de KI:** Escanea ~/.jeff/knowledge/ en busca de archivos metadata.json. Si encuentra aprendizajes previos, inyecta resúmenes de una línea de estos conceptos en el prompt.**Definiciones de artefactos:** Le dice al agente exactamente dónde se encuentra su memoria de trabajo (~/.jeff/brain/<session-id>/) para la sesión actual.**Directivas de comportamiento:** Añade un conjunto de reglas estricto de <planningmode> y un menú de <slashcommands> al prompt del sistema.

2. Comandos de barra (Slash Commands)

La extensión registra comandos interactivos personalizados usando `pi.registerCommand`, y también utiliza comandos nativos para guiar al agente:

- `/plan [tarea] ->` Indica al agente que genere un `implementation_plan.md` en el directorio del cerebro para la tarea especificada.
- `/distill ->` Instruye al agente para que extraiga los aprendizajes clave de la sesión actual y los escriba en un nuevo Elemento de conocimiento.
- `/grill-me ->` Solicita al agente que entre en un modo de entrevista interactiva para aclarar requisitos poco especificados antes de escribir código.
- `/goal [objetivo] ->` Instruye al agente para que trabaje de forma autónoma hacia un objetivo a largo plazo sin detenerse hasta lograrlo por completo.
- `/schedule [tiempo] [instrucción] ->` Establece un horario recurrente o un temporizador único para que el agente ejecute una instrucción específica en segundo plano.

3. Estructura de directorios

La extensión gestiona automáticamente los siguientes directorios:

`~/jeff/brain/<session-id>/ - Memoria de trabajo efímera por`

utilizando y verificar su trabajo de forma independiente. Durante la fase de “Verificación” del flujo de trabajo, el agente: Utilizará la herramienta `bash` para ejecutar la suite de pruebas nativa de su proyecto (ya sea `npm run test`, `pytest`, `cargo test`, `go test`, etc.). Ejecutará comprobaciones de compilación o verificadores de tipos específicos para su lenguaje. Si ocurre un error, la salida se devuelve directamente a la ventana de contexto del agente. Este leerá el seguimiento de la pila, analizará el fallo y utilizará sus herramientas de edición para iterar y corregir el error de forma autónoma. ¿Cómo puedo evitar que el agente se quede atrapado en un bucle? Ocasionalmente, un LLM puede quedar atrapado en un bucle (por ejemplo, aplicando repetidamente la misma corrección que no resuelve un fallo en las pruebas). Aquí le mostramos cómo gestionar esto: **Interrumpir y dirigir:** Usted siempre tiene el control. Si ve que el agente entra en bucle, pulse `Ctrl+C` en su terminal para interrumpir la ejecución de su herramienta. Entonces puede dar una pista como “Estás en un bucle. El problema está en realidad en el esquema de la base de datos, no en el manejador de rutas.” Compactación de contexto: `pi` comprime el contexto de forma nativa. Si un bucle dura demasiado, el contexto antiguo se descarta, lo que a veces puede romper el bucle orgánicamente. Entrevistas interactivas: Si prevé una sesión de depuración compleja, use `/grill-me` de antemano para asegurarse de que el agente comprenda completamente los casos extremos antes de escribir el código, lo que reduce significativamente la posibilidad de una depuración cíclica.

cuenta del error y emite una segunda llamada a la herramienta de edición para corregir el error antes de finalizar su turno.

Usted (Usuario):

¡Solo ve el código final y perfectamente funcional, sin saber que el agente fue forzado a corregir sus propios errores en segundo plano!

FAQ: Gestionando el agente

¿Cómo apruebo o modifico un plan para que el agente comience a desarrollar? Debido a que las reglas de `<planningmode>` inyectadas instruyen al agente a “Obtener la aprobación del usuario” antes de proceder a la ejecución, el agente se detendrá de forma natural y esperará después de terminar de escribir el `implementationplan.md`. Para aprobarlo, simplemente escriba una respuesta en la terminal como:

“Se ve bien, adelante, ejecuta.” Si desea modificar el plan antes de la ejecución: ¡Solo dígame al agente qué cambiar! Debido a que el agente está pausado esperando sus comentarios, puede responder con:

“Antes de ejecutar, cambia el plan para usar `localStorage` en lugar de una base de datos.” “No me gusta el paso 2, reescribamos esa parte para usar una animación CSS en su lugar.” El agente leerá sus comentarios, actualizará el artefacto `implementation_plan.md` con el nuevo enfoque y volverá a hacer una pausa para su aprobación final. Cuando esté satisfecho, diga “Aprobado” y entrará en la fase de “Ejecución”.

¿Cómo va a depurar el agente lo que ha desarrollado? El agente es totalmente independiente del lenguaje y del framework. Aprovecha el conjunto de herramientas nativo de `pi` para inspeccionar su repositorio, averiguar qué stack está

sesión (listas de tareas, planes, tutoriales). `~/jeff/knowledge/`
- Memoria persistente a largo plazo entre sesiones (decisiones de diseño, corrección de errores, reglas de la base de código). **4. Orquestación del arnés**

(Autocorrección, recordatorios JIT y exploración

BFS) Jeff actúa como un orquestador avanzado que hace cumplir estrictamente la calidad y el contexto durante la ejecución:

El bucle de verificación (Puerta de fase de doble capa):

Siempre que el agente modifica el código, Jeff intercepta la salida de la herramienta. Primero, busca un script `.pi-verify.sh` para ejecutar comprobaciones personalizadas del proyecto. Si no lo encuentra, recurre a linters de lenguaje de configuración cero (como `ruff` para Python o `tidy` para HTML). Si alguna comprobación falla, los registros son devueltos directamente al agente como un error crítico, obligándolo a autocorregirse y solucionar los errores antes de continuar. **Recordatorios de contexto Just-In-Time**

(JIT): En lugar de sobrecargar el prompt del sistema con manuales masivos, Jeff inyecta dinámicamente micro-recordatorios específicos en la ventana de atención del agente precisamente cuando los necesita. Por ejemplo, al editar un archivo `.ts`, el agente recibe un recordatorio inmediato para evitar importaciones en línea y funciones auxiliares de una sola línea. **Enrutamiento de modelos**

específico de tareas (Delegación SLM): Jeff cambia dinámicamente el LLM activo según el tipo de tarea. Las tareas complejas (codificación, depuración) utilizan su modelo principal pesado, mientras que las tareas más simples (como los comandos `/plan` o `/distill`) se enrutan automáticamente a Modelos de Lenguaje Pequeño (SLMs) más rápidos y baratos configurados en `jeffmodelrouting.md`. **Exploración BFS del grafo de**

código: Jeff altera fundamentalmente cómo el agente explora la base de código al inyectar la restricción de “Arqueólogo de base de código”. Al agente se le prohíbe estrictamente leer archivos completos de más de 50 líneas de forma lineal. En su lugar, debe rastrear las dependencias

utilizando cortes a nivel de línea y documentar su hipótesis internamente antes de ejecutar lecturas.Herramienta get_outline: Jeff registra una herramienta personalizada que actúa como un analizador estructural. Permite al agente extraer el esqueleto de un archivo (clases, funciones, interfaces) sin contaminar su ventana de contexto con el código fuente, lo que permite lecturas quirúrgicamente precisas.**Ejemplo de uso**Una vez instalado en ~/pi/agent/extensions/jeff.ts, la extensión se ejecuta automáticamente cada vez que inicia pi.

1. Iniciando una tarea compleja

Si le pide a pi que implemente una función importante, las reglas del Modo de planificación inyectadas le impedirán escribir código de inmediato. En su lugar, generará un plan.

Usted:

“Refactoriza el flujo de autenticación para usar JWT en lugar de sesiones.”

Jeff (Agente):

“Debido a que este es un cambio arquitectónico importante, estoy entrando en el Modo de planificación. Primero investigaré la base de código y crearé un implementation_plan.md en ~/.jeff/brain/<session-id>/ para su revisión.”

Puede dirigir manualmente al agente usando los comandos de barra registrados en la terminal interactiva.

Usted:

2. Usando comandos de barra

/plan añadir una nueva capa de caché Redis a la API

Jeff (Agente):

Crea un implementation_plan.md en su directorio cerebral y solicita aprobación.

Usted:

/distill

Jeff (Agente):

Analiza la conversación y escribe un nuevo archivo metadata.json en ~/.jeff/knowledge/redis-caching-pattern/ resumiendo cómo debe manejarse el almacenamiento en caché en este proyecto específico.

3. Siguiendo sesión

La próxima vez que inicie pi, el hook beforeagentstart de Jeff leerá el KI redis-caching-pattern e inyectará su resumen en el prompt del sistema. El agente entenderá de forma nativa sus reglas de caché sin que usted necesite repetirlas.

4. Experimentando el bucle de verificación

(Autocorrección)Si el agente comete un error al escribir código, es posible que ni siquiera lo note. El arnés lo detecta primero:

Jeff (Agente):

Usa la herramienta de edición para modificar un archivo Python, pero introduce accidentalmente un error de sintaxis. Extensión Jeff (Arnés):

Intercepta la edición, ejecuta el linter de lenguaje de forma silenciosa en segundo plano. La comprobación falla. Inyecta un error directamente en el contexto del agente: [Error en puerta de fase del verificador del arnés] Su edición de código Python falló en las comprobaciones de sintaxis. DEBE corregir este error antes de proceder.

Jeff (Agente):

Lee el seguimiento de la pila del error inyectado, se da